Prime-Indexed Sparse State Algebra (PISSA)

A two-layer indexing framework for structured quantum simulation and exact multiplicity diagnostics.

Abstract

Sparse and structured quantum models (lattice systems, stabilizer Hamiltonians, constraint-augmented ensembles) require reliable term management, fast Hamiltonian–vector products, and interpretable structural diagnostics. We introduce **Prime-Indexed Sparse State Algebra (PISSA)**, a **two-layer** architecture separating (i) **state indexing** (squarefree/bitset signatures defining the computational basis) from (ii) **operator indexing** (collision-free keys for templates and placements). This separation provably avoids unintended Hilbert-space enlargement while enabling cache-friendly Hamiltonian application and exact multiplicity statistics via Boolean-lattice transforms. We provide correctness statements (encoding isometry, template/placement decomposition, subset Möbius inversion), implementation patterns (typed masks, template libraries), and a benchmark suite targeting Pauli-string-heavy, constraint-heavy, and repeated-template regimes.

1. Introduction

1.1 Motivation

Many quantum simulation workloads are dominated by repeated application of structured operators (e.g., sums of local terms) to states. Practical bottlenecks include:

- term identity and reproducibility (collision-free naming, serialization),
- template reuse (avoid recomputing local actions),
- dependency tracking on hypergraphs (supports of local terms),
- constraint validation and projection (restricted subspaces; penalty terms),
- structural/multiplicity diagnostics (counts of substructures; parity correlations).

1.2 Core design principle

PISSA treats “prime/bit signatures” strictly as **indexing and bookkeeping**, not as a replacement for linear superposition. A critical safeguard is the separation of:

- **state-indexing bits** defining the Hilbert basis, and
- **operator/constraint metadata keys** used for caching and diagnostics.

1.3 Contributions

1. **Two-layer architecture** with a strict separation of state indices and operator keys.
2. **Template/placement operator representation** with collision-free IDs.
3. **Constraint layers** as either validity-restricted subspaces or penalty Hamiltonians; optional stabilizer-code mode.

4. **Exact multiplicity diagnostics** via subset-lattice zeta/Möbius inversion and parity diagnostics via Walsh–Hadamard transforms.
5. **Algorithms + benchmark suite** clarifying regimes of benefit and limitations.

1.4 Scope and limitations

PISSA is an engineering/tool layer: it does not claim new physics or asymptotic speedups beyond exploiting locality, sparsity, and repeated templates. Worst-case exponential state complexity remains.

1.5 Related work and positioning

PISSA's contribution is primarily architectural: it makes a strict distinction between *basis indexing* and *operator/metadata indexing* while leveraging standard bitmask techniques and well-known transforms.

Exact diagonalization and sparse simulation toolchains. Bitmask-indexed computational bases and Lanczos/ED workflows are standard in many-body physics; see early work on efficient indexing/hashing strategies for spin models [10], and modern open-source packages such as QuSpin [11] and QuTiP [12]. PISSA is compatible with these approaches, but emphasizes canonical term/template organization and collision-free placement identity.

Stabilizers and Pauli/symplectic representations. Efficient manipulation of Pauli operators, stabilizer codes, and Clifford circuits is classical material in quantum information [6–9]. PISSA's Pauli-string apply algorithm is consistent with these conventions, while its constraint layer can be instantiated as either classical validity restrictions or stabilizer-code machinery.

Tensor networks and entanglement-structured simulation. Complementary approaches exploit low entanglement and locality using tensor networks (e.g., MPS/DMRG/PEPS); see Vidal's entanglement-structured simulation perspective [14] and reviews of MPS/DMRG and tensor networks [15,16]. PISSA targets a different axis of structure: *term sparsity, template repetition, and fast structural queries*.

Incidence algebras, Möbius inversion, and fast subset transforms. The subset-lattice Möbius inversion used for multiplicity diagnostics is classical in incidence algebras [1,2]. Fast transforms and related subset convolution algorithms are widely used in exact/parameterized algorithms [3,4], and Walsh–Hadamard/ Fourier analysis on the Boolean cube is a standard toolset for parity/correlation diagnostics [5].

Hamiltonian model tooling for chemistry and fermions. Structured operator sums appearing in quantum chemistry and fermionic simulation pipelines motivate robust term management; see OpenFermion for a representative framework that generates large operator sums with repeated local motifs [13].

2. Preliminaries and Notation

2.1 Sites, configurations, and computational basis

- Sites: $[n] = \{1, \ldots, n\}$.
- Local alphabets: A_k (qubits: $A_k = \{0, 1\}$).
- Configuration space: $X = \prod_{k=1}^{n} A_k$.

- Standard basis: $\{|x\rangle : x \in X\}$.

2.2 Bitmasks and subset operations

Fix m state-index bits. Identify subsets $S \subseteq [m]$ with bitmasks $b \in \{0,1\}^m$ (machine integer representation).

- Bitwise AND, OR, XOR: &,|,\oplus.
- Popcount: $|b|$ (Hamming weight).
- Restriction/extraction operators for subsets of bit positions.

2.3 Boolean lattice and incidence algebra

Let $(\mathcal{P}([m]), \subseteq)$ be the Boolean poset.

- **Zeta transform:** $(\zeta f)(A) = \sum_{B \subseteq A} f(B)$.
- **Möbius transform:** $(\mu g)(A) = \sum_{B \subseteq A} (-1)^{|A \setminus B|} g(B)$. Then μ is the inverse of ζ.

(Optionally) **Walsh–Hadamard transform** on $\{0,1\}^m$ for parity/Fourier analysis.

3. Definitions: PISSA Two-Layer Architecture

Definition 3.1 (State-index prime/bit families)

Choose disjoint state-index families P_k (equivalently disjoint bit ranges) for each site k. Fix injections $\pi_k : A_k \to P_k$. Define the **squarefree signature**

$$ N(x)=\prod_{k=1}^n \pi_k(x_k),\qquad x\in X. $$

Let $S = \{N(x) : x \in X\}$. In implementations, store $N(x)$ as a **StateKey** bitmask over $P_{\text{state}} = \bigsqcup_k P_k$.

Definition 3.2 (State space)

Define

$$ \mathcal H:=\ell^2(S),\quad \langle f,g\rangle=\sum_{N\in S} \overline{f(N)}g(N). $$

The computational basis is $\{\delta_{N(x)}\}_{x \in X}$.

Definition 3.3 (Operator/signature layer)

Let E be a set of supports (hyperedges) $e \subseteq [n]$. Assign collision-free **TermIDs** (conceptually primes) to:

- templates (operator patterns), and/or
- placements (template + support).

Rule (No mixing): TermIDs and syndrome records are **not** embedded into StateKeys.

Definition 3.4 (Templates and placements)

A **template** τ is a k-local operator acting on $\bigotimes_{i=1}^{k} \mathbb{C}^{|A_{s_i}|}$ (represented as a matrix or an apply routine). A **placement** is a pair (τ, e) with $|e| = k$, together with an embedding ι_e placing τ into the global space.

Definition 3.5 (PISSA Hamiltonian)

A PISSA Hamiltonian is represented as

$$ H=\sum_{j=1}^M c_j\,\iota_{e_j}(\tau_{t(j)}), $$

where $t(j)$ indexes a finite template library and e_j specifies placement support.

Definition 3.6 (Constraint layers)

Two nonexclusive modes:

1. **Validity restriction:** choose $S_{\text{valid}} \subseteq S$, work on $\mathcal{H}_{\text{valid}} = \ell^2(S_{\text{valid}})$.
2. **Penalty constraints:** add $H_{\text{pen}} = \lambda \sum_{\ell} \Pi_{\ell}$ where Π_{ℓ} projects onto violating basis states.

(Optional) Stabilizer-code mode: commuting Pauli stabilizers with syndromes stored separately.

4. Correctness Statements

Theorem 4.1 (Encoding isometry)

Statement. Define $U : \mathbb{C}^{|X|} \to \ell^2(S)$ by $U|x\rangle = \delta_{N(x)}$. Then U is unitary. Equivalently, the encoding is a basis permutation under the bijection $x \leftrightarrow N(x)$.

Proof sketch. Disjointness of prime/bit families makes $x \mapsto N(x)$ injective; finite sets imply bijection onto S. Orthonormality is preserved.

Proposition 4.2 (No Hilbert space enlargement)

Statement. If TermIDs and syndromes are stored outside StateKeys, then $\dim \mathcal{H} = |X|$ is independent of the number of operator keys $|Q|$ and syndrome tags $|R|$.

Theorem 4.3 (Template/placement decomposition)

Statement. Any Hamiltonian written as a sum of local terms $\{H_e\}_{e \in E}$ admits a representation

$$ H=\sum_{j=1}^M c_j\,\iota_{e_j}(\tau_{t(j)}) $$

where $\{\tau_i\}_{i=1}^{r}$ is a finite template library obtained by grouping terms into equivalence classes under a chosen canonical form.

Remark. With a fixed canonicalization convention (basis ordering, site relabeling, normalization), template IDs can be made deterministic.

Theorem 4.4 (Subset Möbius inversion correctness)

Statement. For any $f : \mathcal{P}([m]) \to \mathbb{C}$, the zeta transform $g = \zeta f$ is invertible with inverse $f = \mu g$:

$$ g(A)=\sum_{B\subseteq A} f(B),\qquad f(A)=\sum_{B\subseteq A} (-1)^{|A\setminus B|} g(B). $$

Corollary 4.5 (Squarefree signature interpretation)

Statement. When StateKeys correspond to squarefree signatures, subset inclusion corresponds to divisibility of labels; Boolean-lattice Möbius inversion recovers integer Möbius inversion restricted to squarefree supports.

5. Algorithms

This section gives pseudocode without committing to any programming language. Conventions:

- `StateKey` is a bitmask over state-index bits only.
- `SupportMask` is a bitmask over state-index bits.
- Sparse states are maps `psi : StateKey -> complex`.
- Hamiltonians are lists of terms, each term stores `(template_id, support, coeff, placement_meta)`.

5.1 Algorithm I: Apply a Pauli-string term

Assume a Pauli-string template is represented by:

- `flip_mask` (bits to XOR),
- `phase_mask` (bits contributing a (-1) phase),
- optional `i_phase_rule` for Y operators (implementation-specific),
- scalar coefficient `c`.

Goal: compute `phi += c * P * psi`.

```
Algorithm ApplyPauliTerm(psi, coeff c, flip_mask, phase_mask, y_mask_optional):
    phi := empty map StateKey -> complex (or reuse accumulator)
    for each (s, amp) in psi:
        s2 := s XOR flip_mask

        # real phase from parity of intersection
        parity := POPCOUNT(s AND phase_mask) mod 2
        phase := (+1 if parity == 0 else -1)

        # optional: handle Y phases (convention-dependent)
```

```
        # Example placeholder: phase *= (i)^(k(s)) where k depends on bits in
y_mask_optional
        if y_mask_optional is present:
            phase := phase * YPhaseFactor(s, y_mask_optional)

        phi[s2] := phi[s2] + c * phase * amp

    return phi
```

Notes.

- For many Pauli terms, fuse loops by streaming through `psi` once per term or batching terms by shared masks.
- `YPhaseFactor` must match a chosen Pauli convention; it can be implemented via precomputed tables on local blocks.

5.2 Algorithm II: Apply a dense k-local term (gather–multiply–scatter)

Assume a term acts on a support set `sites = [i1,...,ik]` with a dense local matrix `M` of size $d^k \times d^k$ (qubits: $d = 2$). The action is

$$ \phi = \phi + c\,\iota_e(M)\psi. $$

Key operations:

- `ExtractLocalIndex(s, sites)` returns $\ell \in \{0, \ldots, d^k - 1\}$.
- `ReplaceLocalIndex(s, sites, ell)` returns new global key with local bits replaced.

```
Algorithm ApplyDenseKLocalTerm(psi, coeff c, sites[1..k], local_matrix M):
    phi := empty map StateKey -> complex (or reuse accumulator)

    # Group amplitudes by the "outer" bits (bits not in support)
    groups := map OuterKey -> vector block[0..d^k-1]

    for each (s, amp) in psi:
        outer := RemoveBits(s, sites)             # key for bits outside the
support
        ell   := ExtractLocalIndex(s, sites)      # local basis index
        groups[outer][ell] := groups[outer][ell] + amp

    for each outer in groups:
        v := groups[outer]                        # length d^k
        w := M * v                                # matrix-vector multiply
        for ell2 in 0..d^k-1:
            if w[ell2] != 0:
                s2 := InsertBits(outer, sites, ell2)
```

```
                phi[s2] := phi[s2] + c * w[ell2]

    return phi
```

Notes.

- For sparse `psi`, `groups` may remain sparse; implement blocks as small arrays with only filled entries.
- Cache `ExtractLocalIndex` / `InsertBits` maps per placement for speed.
- If k is small (2–4), this is often practical; for large k, cost scales as d^k.

5.3 Algorithm III: Fast zeta and Möbius transforms on the Boolean cube

Assume a function `f` on subsets of $[m]$ represented as an array `F[0..2^m-1]` indexed by bitmasks.

Fast zeta transform

Computes `G[A] = sum_{B subseteq A} F[B]`.

```
Algorithm FastZetaTransform(F, m):
    G := copy(F)
    for i in 0..m-1:
        for mask in 0..2^m-1:
            if (mask has bit i set):
                G[mask] := G[mask] + G[mask without bit i]
    return G
```

Fast Möbius transform

Inverse of zeta; computes `F` from `G`.

```
Algorithm FastMobiusTransform(G, m):
    F := copy(G)
    for i in 0..m-1:
        for mask in 0..2^m-1:
            if (mask has bit i set):
                F[mask] := F[mask] - F[mask without bit i]
    return F
```

Notes.

- Complexity: $O(m2^m)$. Useful for diagnostics on small/medium m or on reduced subsystems.
- For large systems, use partial transforms on selected bit ranges or sparse variants.

6. Implementation Notes

6.1 Typed masks (semantic safety)

Use the same underlying integer type but enforce conventions:

- `StateKey`: only state-index bits.
- `SupportMask`: only state-index bits.
- `TemplateID`, `PlacementID`: integers (or collision-free prime keys) *not OR'd* into StateKeys.
- `Syndrome`: stored alongside the state vector, not inside StateKeys.

6.2 Template libraries and canonicalization

- Canonicalize local operators (basis ordering, normalization) to ensure deterministic template IDs.
- Cache local matrices or apply-routines per template.

6.3 Placement metadata caching

For each placement support `e` cache:

- extraction/insertion maps,
- bit position lists,
- (optional) precomputed lookup tables for Pauli phases on small supports.

6.4 Operator application strategies

- Group terms by template to maximize reuse.
- For Pauli-heavy Hamiltonians, fuse application loops or batch terms.
- For diagonal constraints, apply in-place without gather/scatter.

6.5 Constraint modes

- Validity restriction: reject invalid inserts; project by deletion/renormalization.
- Penalty constraints: represent as diagonal projectors.
- Stabilizer mode: represent stabilizers as Pauli templates; compute syndromes via parity logic.

6.6 Serialization and reproducibility

- Serialize template library + placements + coefficients.
- Collision-free keys (conceptual primes) provide stable term naming.

7. Benchmark Suite

7.1 Hypotheses

- **H1:** Template caching reduces constant factors for repeated local structures.
- **H2:** Pauli-string application is especially efficient (bitwise XOR + parity phase).
- **H3:** Constraint-heavy workflows benefit from O(1) validity checks and diagonal penalties.

- **H4:** Boolean-lattice transforms provide exact multiplicity diagnostics on small/medium subsystems.

7.2 Workloads

1. Random sparse Pauli Hamiltonians (fixed locality k, varying term count M).
2. Translationally invariant lattice models (TFIM, Heisenberg variants) with repeated templates.
3. Constraint-heavy models (particle number, parity, projector-augmented Hamiltonians).
4. Stabilizer Hamiltonians / code checks (optional): syndrome extraction and recovery.

7.3 Baselines

- Conventional sparse operator storage without template grouping.
- Standard bitmask Pauli application without PISSA term-key/template organization.
- Dense methods where feasible (small n) for correctness checks.

7.4 Metrics

- Runtime per $H\psi$ and per evolution step/iteration.
- Memory: templates cached, placement metadata, state nnz.
- Scaling: n, M, locality k, $\mathrm{nnz}(\psi)$.
- Correctness: regression vs brute force (small n), norm/energy checks.

7.5 Reporting

- Plots: runtime vs n, runtime vs M, runtime vs nnz.
- Regime map: where PISSA wins/loses (Pauli/diagonal vs dense high-k).

8. Discussion and Limitations

- No new physics; benefits are engineering + diagnostics.
- Worst-case exponential state complexity remains.
- Dense high-k terms reduce gains.
- Transforms are $O(m2^m)$; use on subsystems or diagnostic slices.

9. Conclusion

PISSA provides a mathematically clean and implementable two-layer indexing system for structured quantum simulation, with provable correctness properties and practical benefits in caching, constraint handling, and multi

plicity diagnostics.

References

[1] G.-C. Rota, “On the Foundations of Combinatorial Theory I: Theory of Möbius Functions,” *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* **2**, 340–368 (1964). DOI: 10.1007/BF00531932.

[2] T. Leinster, “Notions of Möbius inversion,” arXiv:1201.0413 (2012).

[3] A. Björklund, T. Husfeldt, and P. Koivisto, “Fourier Meets Möbius: Fast Subset Convolution,” in *Proceedings of STOC* (2007). DOI: 10.1145/1250790.1250801.

[4] M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk, M. Pilipczuk, and S. Saurabh, *Parameterized Algorithms*, Springer (2015/2016). DOI: 10.1007/978-3-319-21275-3.

[5] R. O’Donnell, *Analysis of Boolean Functions*, Cambridge University Press (2014). (Freely available revision: arXiv:2105.10386, 2021.)

[6] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press (2000).

[7] D. Gottesman, “Stabilizer Codes and Quantum Error Correction,” Ph.D. thesis, Caltech (1997). arXiv\:quant-ph/9705052.

[8] S. Aaronson and D. Gottesman, “Improved Simulation of Stabilizer Circuits,” *Phys. Rev. A* **70**, 052328 (2004). DOI: 10.1103/PhysRevA.70.052328.

[9] J. Dehaene and B. De Moor, “The Clifford group, stabilizer states, and linear and quadratic operations over GF(2),” *Phys. Rev. A* **68**, 042318 (2003). DOI: 10.1103/PhysRevA.68.042318.

[10] H. Q. Lin, “Exact diagonalization of quantum-spin models,” *Phys. Rev. B* **42**, 6561 (1990). DOI: 10.1103/PhysRevB.42.6561.

[11] P. Weinberg and M. Bukov, “QuSpin: a Python package for dynamics and exact diagonalisation of quantum many body systems. Part I: spin chains,” *SciPost Phys.* **2**, 003 (2017). arXiv:1610.03042.

[12] J. R. Johansson, P. D. Nation, and F. Nori, “QuTiP 2: A Python framework for the dynamics of open quantum systems,” *Comput. Phys. Commun.* **184**, 1234–1240 (2013). arXiv:1211.6518.

[13] J. R. McClean *et al*., “OpenFermion: The Electronic Structure Package for Quantum Computers,” arXiv: 1710.07629 (2017).

[14] G. Vidal, “Efficient classical simulation of slightly entangled quantum computations,” *Phys. Rev. Lett.* **91**, 147902 (2003). arXiv\:quant-ph/0301063.

[15] U. Schollwöck, “The density-matrix renormalization group in the age of matrix product states,” *Annals of Physics* **326**, 96–192 (2011). arXiv:1008.3477.

[16] R. Orús, “A Practical Introduction to Tensor Networks: Matrix Product States and Projected Entangled Pair States,” *Annals of Physics* **349**, 117–158 (2014). arXiv:1306.2164.